

```

import cv2
import numpy as np
import hashlib

face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
)
eye_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_eye.xml'
)

def read_image_bytes(image_bytes):
    np_arr = np.frombuffer(image_bytes, np.uint8)
    return cv2.imdecode(np_arr, cv2.IMREAD_COLOR)

def blur_score(img):
    """Laplacian variance - higher = sharper"""
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    variance = cv2.Laplacian(gray, cv2.CV_64F).var()
    # Also check local blur using tiled regions
    h, w = gray.shape
    tile_scores = []
    for r in range(0, h, h//3):
        for c in range(0, w, w//3):
            tile = gray[r:r+h//3, c:c+w//3]
            tile_scores.append(cv2.Laplacian(tile, cv2.CV_64F).var())
    local_min = min(tile_scores)
    # Penalize if any region is very blurry
    penalty = 0 if local_min > 50 else (50 - local_min) / 10
    score = min(variance / 10, 30) - penalty
    return max(0, score)

def brightness_score(img):
    """Ideal brightness around 120-150 range"""
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    v = hsv[:, :, 2]
    brightness = v.mean()
    # Also check if image is overexposed or underexposed in large areas
    overexposed = np.sum(v > 240) / v.size
    underexposed = np.sum(v < 15) / v.size
    penalty = (overexposed + underexposed) * 20
    score = max(0, 20 - abs(brightness - 135) / 6) - penalty
    return max(0, score)

def contrast_score(img):
    """RMS contrast - more accurate than std alone"""
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float32)
    rms = np.sqrt(np.mean((gray - gray.mean()) ** 2))
    # Check histogram spread
    hist = cv2.calcHist([cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)],
                        [0], None, [256], [0, 256])
    hist = hist.flatten() / hist.sum()
    spread = np.sum(hist > 0.001) # how many bins are used
    spread_bonus = min(spread / 256 * 10, 5)
    return min(rms / 3 + spread_bonus, 20)

def noise_score(img):
    """Penalize noisy images"""
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float32)
    # Estimate noise using high-frequency residual
    blur = cv2.GaussianBlur(gray, (5, 5), 0)
    noise = np.std(gray - blur)
    # Low noise = good score
    score = max(0, 10 - noise / 2)
    return min(score, 10)

def composition_score(img):
    """Basic rule of thirds + edge density"""
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    h, w = gray.shape
    edges = cv2.Canny(gray, 50, 150)
    # Rule of thirds points
    thirds_points = [
        (h//3, w//3), (h//3, 2*w//3),
        (2*h//3, w//3), (2*h//3, 2*w//3)
    ]

```

```

J
margin = min(h, w) // 10
thirds_score = 0
for (r, c) in thirds_points:
    region = edges[max(0,r-margin):r+margin, max(0,c-margin):c+margin]
    if region.mean() > 10:
        thirds_score += 1.5
return min(thirds_score, 5)

def face_bonus(img):
    """Detect faces + eyes for better accuracy"""
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(
        gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30)
    )
    if len(faces) == 0:
        return 0
    bonus = 0
    for (x, y, fw, fh) in faces:
        face_region = gray[y:y+fh, x:x+fw]
        eyes = eye_cascade.detectMultiScale(face_region, 1.1, 3)
        # Full face with eyes = more bonus
        if len(eyes) >= 2:
            bonus += 15
        else:
            bonus += 8
    return min(bonus, 20)

def image_hash(img):
    """Perceptual hash for duplicate detection"""
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = cv2.resize(gray, (16, 16)) # larger = more accurate
    avg = gray.mean()
    bits = ''.join('1' if p > avg else '0' for p in gray.flatten())
    return hashlib.sha256(bits.encode()).hexdigest()

def calculate_score(img, duplicate_hashes):
    blur = blur_score(img) # max 30
    brightness = brightness_score(img) # max 20
    contrast = contrast_score(img) # max 20
    noise = noise_score(img) # max 10
    composition = composition_score(img) # max 5
    face = face_bonus(img) # max 20

    img_hash = image_hash(img)
    duplicate_penalty = 30 if img_hash in duplicate_hashes else 0

    total = blur + brightness + contrast + noise + composition + face
    total = total - duplicate_penalty
    total = max(0, min(total, 100))

    return total, img_hash

def enhance_image_basic(img):
    """OpenCV fallback enhancement"""
    img = cv2.fastNlMeansDenoisingColored(img, None, 10, 10, 7, 21)
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
    cl = clahe.apply(l)
    img = cv2.cvtColor(cv2.merge((cl, a, b)), cv2.COLOR_LAB2BGR)
    kernel = np.array([[0, -1, 0],
                       [-1, 5, -1],
                       [0, -1, 0]])
    return cv2.filter2D(img, -1, kernel)

```

